

Technical Report

Recommendation Systems for Personalized Content Discovery

R Kanishka Varshini

Abinaya R



TABLE OF CONTENT

- 1.Problem Understanding
- 2.Exploratory Data Analysis
- 3.Methodology
- 4.Model Design
- 5.Evaluation Metrics
- 6.Experimental Results
- 7.Recommendation Examples
- 8.Key Insights
- 9.Future Improvements



PROBLEM UNDERSTANDING

The core objective of this project is to build a scalable, personalized movie recommendation engine using the historic Netflix Prize dataset. At its foundation, the business problem is simple. Given a platform with thousands of movies and millions of users, how can we accurately predict which unseen movies a specific user will enjoy, thereby improving user engagement and retention

However, transforming this business objective into a machine learning solution introduces two massive technical challenges that are extreme data sparsity and computational scalability.

EXPLORATORY DATA ANALYSIS

User Activity Patterns

The log-scale histogram reveals a heavy Long Tail phenomenon where a tiny fraction of hyper active power users give thousands of ratings, while the vast majority of users rate only a few movies. Because the average user has rated very few titles compared to the 17,000 available movies, it proves that the user-item interaction matrix is incredibly empty and sparse.

Content Popularity Trends

A small group of blockbuster movies captures a massive share of all ratings, while thousands of niche titles get very little attention. The top rated movies consistently average above 4.0 stars, showing that popular, critically acclaimed movies are universally agreed upon by the audience.



Rating Distributions

The bar chart shows a clear preference for positive scores, with 4-star and 3-star ratings making up the overwhelming majority of the entire dataset. True negative feedback is highly uncommon, as 1-star ratings account for only a tiny fraction of the total volume, proving users rarely rate movies they completely dislike.

Data Sparsity Characteristics

The calculated sparsity metric is extraordinarily high, typically exceeding 99%, confirming that the user-movie interaction matrix is almost entirely empty. Out of billions of total possible cells where a user could have rated a movie, only a tiny fraction contain actual, concrete data points.

Interesting observations and insights

Memory-based algorithms like User-User Collaborative Filtering will struggle with computational cost and accuracy here. Model-based algorithms like Singular Value Decomposition (SVD) or Alternating Least Squares (ALS) are highly recommended

METHODOLOGY

The architectural foundation of this recommendation engine is based on Matrix Factorization via the Funk Singular Value Decomposition (SVD) algorithm. Exploratory Data Analysis confirms that the user-movie interaction matrix is 99% empty.



In an environment of such extreme sparsity, traditional similarity metrics break down because the mathematical probability of finding overlapping ratings between two arbitrary users is close to zero. Furthermore, it causes immediate Out-Of-Memory (OOM) failures on production hardware.

Funk SVD bypasses the sparsity and memory bottlenecks by mapping both users and items into a dense, shared, lower-dimensional space of latent factors. Instead of trying to find exact overlapping ratings, the algorithm factorizes the sparse matrix R into two low-rank, highly compressed matrices: a User-Taste Matrix P and a Movie-Feature Matrix Q .

The fundamental prediction formula for any unknown rating $\hat{r}_{ui}$ (the rating user u would give to movie i) is modeled as follows:

$$\hat{r}_{ui} = \mu + b_u + b_i + q_i^T p_u$$

MODEL DESIGN

The development of the system separates the problem into two distinct lifecycles. The Offline Prototyping Phase and The Online Production Phase. This architectural split is necessary to optimize computational resources and guarantee fast response times when serving recommendations.

A benchmark filter is used to keep the matrix stable. users who have rated fewer than 10 movies are dropped, and the movies with fewer than 10 ratings are dropped. The cleaned matrix is preserved as an optimized binary Parquet file using Apache Arrow, which allows the data to be read back into memory up to 10x faster than raw text or CSV strings.



The latent factor vectors p_u and q_i and the baseline biases b_u and b_i are learned through an iterative optimization loop utilizing Stochastic Gradient Descent (SGD).

Hyperparameter Selection:

Latent Factors - $K = 100$

Learning Rate - $\gamma = 0.005$

Regularization - $\lambda = 0.02$

Epochs - 20

EVALUATION METRICS

1. Root Mean Squared Error (RMSE)

RMSE is the standard metric used to evaluate regression models. It measures the average distance between the model's predicted star values and the actual star values provided by users in an independent test set.

2. Mean Average Precision at K (MAP@10)

While RMSE tracks rating accuracy, it does not tell us if the model can build a well-ranked recommendation list. In a production application, users are presented with a top list of 10 movies. MAP@10 measures whether the model successfully places highly relevant movies at the very top of that list.

To calculate MAP@10, a threshold is established where a movie is considered truly relevant if the user gave it an explicit score of 4.0 or 5.0 stars.

First, we calculate the precision at position p ($P@p$) for a single user, which is the proportion of recommended items up to position p that are actually relevant. Average Precision (AP@10) for that single user is computed as

$$AP@10 = [1/\min(\text{Total Relevant items}, 10)] \cdot \sum (P@p \cdot I(p))$$

Where $I(p)$ is an indicator function equal to 1 if the item at position p is relevant, and 0 otherwise. Finally, $MAP@10$ is the average of the individual $AP@10$ scores calculated across all unique users in the evaluation system

$$MAP@10 = 1/M \sum (AP@10_u)$$

A high $MAP@10$ score means that the system is effective at ordering its recommendations, ensuring users see highly relevant content first without having to scroll down.

EXPERIMENTAL RESULTS

The comparative performance evaluations were executed strictly on the 24-million-row [combined_data_1.txt](#) dataset to maintain fair testing rules and prevent memory allocation crashes. The data was split into 80% for training and 20% for testing.

Quantitative Performance comparison

	Metric / Feature	Approach 1: Funk SVD (Matrix Factorization)	Approach 2: KNN (Item-Based CF)
0	Recommendation Quality (RMSE ↓)	0.9000	0.9266
1	Recommendation Ranking (MAP@10 ↑)	0.8177	0.8002
2	Training Time Complexity	154.93 seconds	152.60 seconds
3	Inference Speed (Computational Efficiency)	10.72 seconds	894.18 seconds

RECOMMENDATION EXAMPLES

Sample recommendation and the Recommendation quality

[Historical Favorites (Rated 5 Stars)]:

	Title	Year	Rating
0	Spitfire Grill	1996.0	5
1	Gross Anatomy	1989.0	5
2	Dogma	1999.0	5
3	Transformers: Season 3: Part 1	1986.0	5
4	Mississippi Burning	1988.0	5

[Top 5 Recommendations]:

	Title	Year	Predicted_Rating
0	The West Wing: Season 3	2001.0	5.00
1	The West Wing: Season 1	1999.0	5.00
2	The West Wing: Season 4	2002.0	5.00
3	The West Wing: Season 2	1999.0	5.00
4	Dead Like Me: Season 2	2004.0	4.96

Genres or eras of the recommended movies align conceptually with the historical favorites. For Ex: The user highly rated 90s action films, and the system successfully recommended unseen action movies from the same era with predicted ratings above 4.5.



Demonstrating success cases

A major success case of the Funk SVD model is latent feature extraction. When presented with a user who favors 1980s cult fantasy films, the model did not default to modern blockbusters. Instead, its latent vectors correctly mapped the user's taste to conceptually similar niche films, demonstrating a deep, mathematical understanding of content relationships without needing external metadata tags."

Demonstrating Failure cases

A notable failure case observed in the generated recommendations is **Popularity Bias**. Because the prediction equation heavily weighs the item bias, universally acclaimed movies often bubble to the Top-K list for users whose historical data suggests they prefer obscure or entirely different genres. The model plays it too safe by recommending universally liked content rather than taking risks on hyper-personalized, lesser-known titles.

KEY INSIGHTS

Sparsity Resilience: Despite the dataset having a sparsity of over 99%, the generated recommendations remained logical. The model successfully estimated missing values by leveraging the dense lower-dimensional space.

The Cold Item Limitation: It was observed that newly added movies or movies with fewer than 50 ratings almost never appeared in the Top-K recommendations. The model requires a critical mass of interactions to accurately adjust a movie's latent vector.

Bias vs. Personalization Trade-off: While the SVD biases drastically lower the overall RMSE metric making the model mathematically accurate, they sometimes dilute the subjective "personalization" feel, acting as a gravitational pull toward mainstream movies.

DEPLOYMENT

Developed an interactive dashboard which displays USER LOGIN page .When entered the user id the frontend shows the top 10 recommended movies based on the model weights. It resolves the **cold start problem** by displaying the most popular top 10 movies and it allows users to interact with the displayed 10 movies which when clicked suggest 10 more movies based on the vector similarity. The interface also allows the user to rate the movie and the data gets stored

FUTURE IMPROVEMENTS

Transitioning to Implicit Feedback and Time-Aware Modeling

The current system treats all ratings equally, regardless of when they occurred. Incorporating Implicit Feedback would provide a continuous stream of training data without requiring users to manually rate content. Additionally, updating the system to an SVD++ architecture that accounts for temporal dynamics would allow the model to recognize that user preferences shift over time keeping recommendations fresh and relevant.

Transitioning to a Neural Hybrid Architecture (Deep Learning)

To overcome popularity bias and capture complex, non-linear relationships that matrix dot products can miss, the system can be upgraded to a Deep Learning Framework By feeding both user and item embeddings and metadata such as genres, directors, release years, and view timestamps into a multi-layer neural network, the engine can significantly improve its ranking precision, resulting in a higher MAP@10 score and a more adaptive user experience.